

PVMSim User Manual

PVMSim: A MATLAB App for Reproducible Double-Diode Parameter Extraction from
Measured Photovoltaic Current–Voltage Curves

Software name:	PVMSim
Software version:	0.1.0
Manual version:	1.0
Platform:	MATLAB R2025b
Repository:	https://github.com/pvmsim
Contact:	pvmsim.matlab@gmail.com

July 26, 2026

PVMSim: A MATLAB App for Reproducible Double-Diode Parameter Extraction from Measured Photovoltaic Current–Voltage Curves

Authors

Liomnis Osorio^{a,b,c*}
Mailyn Moreno-Espino^{d,e}
Laurent Duchêne^c
Víctor Tuninetti^f
Carlos Zalazar^b
Rodrigo Irarrázaval^b
Yoalbys Retirado-Mediaceja^{g,h}
Marco Rivera^{i,j}

Affiliations

^a Department of Industrial Processes, Faculty of Engineering, Universidad Católica de Temuco, Temuco, Chile

^b Doctoral Program in Engineering, MacroFaculty of Engineering (UFRO–UBB–UTALCA Consortium), Chile

^c ArGEnCo Department, MSM team, University of Liège, Liège, Belgium

^d Faculty of Informatics, Universidad Complutense de Madrid, Madrid, Spain

^e Institute of Knowledge Technology, Universidad Complutense de Madrid, Madrid, Spain

^f Department of Mechanical Engineering, Universidad de La Frontera, Temuco, Chile

^g Universidad de Moa Dr. Antonio Núñez Jiménez, Moa, Cuba

^h Sección de Ciencias Técnicas, Academia de Ciencias de Cuba, La Habana, Cuba

ⁱ Laboratorio de Conversión de Energías y Electrónica de Potencia (LCEEP), Universidad de Talca, Curicó, Chile

^j Power Electronics, Machines and Control (PEMC) Research Institute, Faculty of Engineering, Nottingham, United Kingdom

PVMSim User Manual

Version 1.0

July 26, 2026

About PVMSim

PVMSim is a MATLAB-based application for parameter extraction of the double-diode photovoltaic model from measured current–voltage (I–V) curves. The software combines a graphical user interface and a scriptable workflow to support staged optimization, model inspection, fit assessment, and structured export of run artifacts. The current version is intended for research, teaching, and reproducible comparative studies involving photovoltaic module characterization from measured I–V data.

Disclaimer

Disclaimer

PVMSim is provided as research software. The application supports parameter extraction, visualization, and export of run information, but the interpretation of fitted parameters remains dependent on input quality, model assumptions, and run settings. The software does not replace expert engineering judgment for field diagnosis, warranty decisions, or bankable photovoltaic performance assessment.

Availability

The source code, documentation, and example files are distributed through the project repository:

<https://github.com/pvmsim>

Please verify that the repository version, MATLAB release, and example data are consistent with the version reported in this manual.

Contents

About PVMSim	i
Disclaimer	i
Availability	i
1 Introduction to PVMSim	1
1.1 Overview	1
1.2 System Requirements	1
1.3 PVMSim GUI Architecture	1
2 PVMSim Settings and Interface Elements	1
2.1 Run controls	1
2.1.1 Step 1A. Save the validated project configuration	2
2.2 Status flags	3
2.3 Tabs and outputs	3
3 Getting Started	4
3.1 Downloading PVMSim	4
3.2 Launching from MATLAB	4
3.3 Verifying the installation with the automated smoke test	4
4 Step-by-Step Tutorial	5
4.1 Single-run workflow	5
4.1.1 Step 1. Set the project workspace and validate the default folders	5
4.1.2 Step 2. Import the PV module electrical data	7
4.1.3 Step 3. Import and select the measured I–V curve	9
4.1.4 Step 4. Review the algorithm tab before execution	12
4.1.5 Step 5. Run the staged parameter extraction	13
4.1.6 Step 6. Inspect plots and metrics after the run	15
4.2 Batch-run workflow	16
5 Input Data and File Formats	18
5.1 Module definition files	18
5.2 Measured I–V files	19
5.3 Optional metadata	19
6 Output Files and Run Exports	19
6.1 log.txt	19
6.2 run_summary.csv	19
6.3 run_results.mat	19
6.4 run_config.json	19
6.5 checksums.sha256	20
7 Recommended Workflow	20
8 Troubleshooting	21
9 Relation to CLI execution	21
10 References	21

1. Introduction to PVMSim

1.1. Overview

PVMSim implements a workflow for fitting the double-diode model (DDM) to measured I–V curves. The user selects a project folder, loads a photovoltaic module definition file, imports measured I–V data, configures the optimization controls, and runs a staged extraction routine. After execution, the application provides convergence information, extracted parameters, fit metrics, curve plots, and optional batch statistics.

The software is designed to support three recurring use cases:

- exploratory single-run parameter extraction from measured I–V curves;
- repeated runs for sensitivity and robustness analysis;
- traceable export of run-level outputs for later inspection and comparison.

1.2. System Requirements

The recommended environment for the current version is listed below.

Table 1. Recommended system requirements for PVMSim.

Item	Requirement
Operating system	Windows 11 (tested). Other MATLAB-supported platforms may also work, but they have not been validated in the current release.
MATLAB release	MATLAB R2025b (tested). Close releases may also work, but they have not been systematically validated.
Toolboxes	No additional MATLAB toolboxes are required for the current repository version.
Display	A screen resolution of 1920×1080 or higher is recommended for comfortable GUI use.
Storage	Free disk space is required for project files, measured I–V datasets, exported figures, run folders, and log files.

1.3. PVMSim GUI Architecture

Figure 1 shows the general layout of the PVMSim graphical user interface. The interface is organized around a persistent workspace and several task-oriented tabs. The persistent left-side area centralizes project selection, loaded file status, module information, and application-level run logging. The main tab region is used for data inspection, algorithm execution, validation plots, metrics, and repeated runs.

2. PVMSim Settings and Interface Elements

2.1. Run controls

PVMSim includes the following main controls for configuring and executing the parameter-extraction workflow:

- **Run:** starts the staged parameter extraction routine.

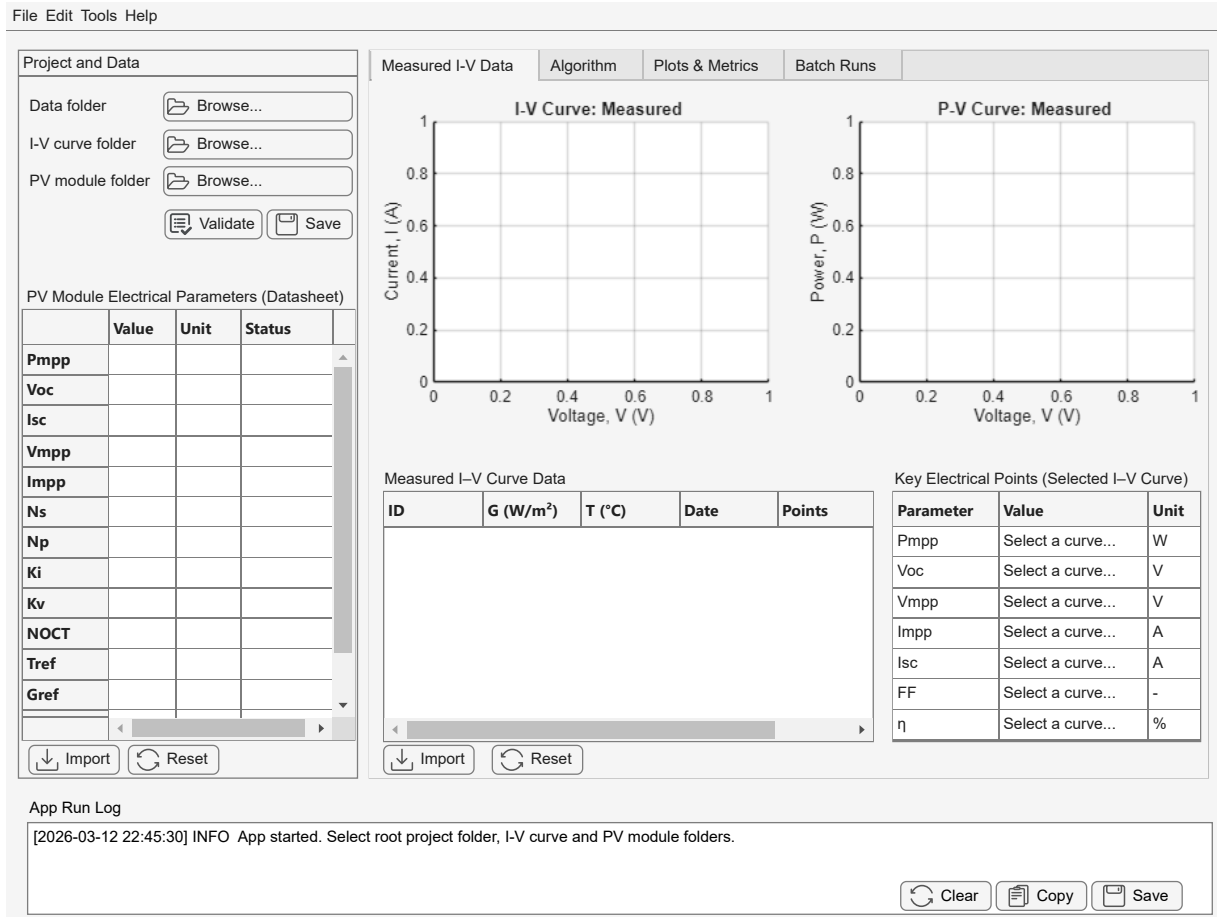


Figure 1. General layout of the PVMSim graphical user interface.

- **Seed:** sets the random seed for controlled reruns.
- **MaxIter:** sets the iteration budget used by the optimization core.
- **Reset/Clear:** clears selected views or restores default states.
- **Export:** writes selected outputs such as figures, logs, or summaries.

2.1.1. Step 1A. Save the validated project configuration

After the workspace has been configured and validated, the **Save** button shown in Figure 2 can be used to store a lightweight project configuration file. This action first checks that the configured paths are valid. The app then opens a file-selection dialog and saves a **.mat** file, typically named **project_config.mat**, containing the current workspace paths, the current values of **Seed** and **MaxIter**, and the save timestamp.

This action is intended to preserve the current project setup for later reuse. It does not export extracted parameters, plots, KPI tables, or batch summaries.

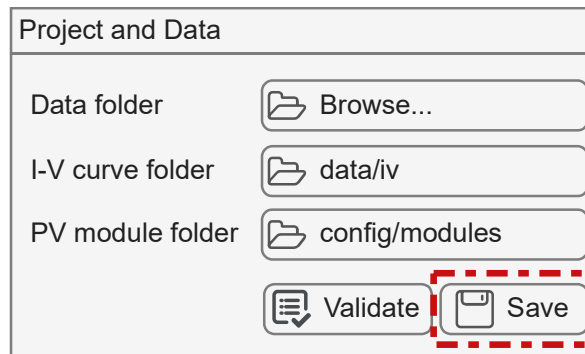


Figure 2. Save button in the Project and Data panel, used to store a validated project configuration.

✓ Expected outcome

After a valid workspace has been configured, the Save button should open a dialog to write a `project_config.mat` file containing the current paths and algorithm-control values.

i Note

The saved configuration preserves the current workspace setup and algorithm controls, but it does not store extraction outputs, imported measured-curve tables, or post-run metrics.

2.2. Status flags

The GUI uses status indicators and validation cues to show whether the workspace is ready, whether the imported module data contain missing entries, and whether the loaded inputs are consistent enough to proceed with execution.

i Note

Status indicators in PVMSim app version reflect whether the configured workspace has been validated, whether imported module entries are available or missing, and whether the execution panels remain in a waiting, running, completed, or failed state.

2.3. Tabs and outputs

The main tabs group the workflow into distinct analysis views:

- **Project and Data**
- **Measured I–V Data**
- **Algorithm**
- **Plots & Metrics**
- **Batch Runs**

Each tab presents a specific stage of the workflow through the corresponding tables, plots, metrics, and repeated-run summaries available in PVMSim app.

3. Getting Started

3.1. Downloading PVMSim

Clone or download the repository from the official project page:

<https://github.com/pvmsim>

Store the project in a dedicated folder that preserves the internal directory structure.

3.2. Launching from MATLAB

Open MATLAB and set the Current Folder to the project root. Then open the subfolder `app/` and double-click `PVMSim.mlapp` to launch the graphical user interface (GUI).

Before opening the app, verify that the repository structure is complete and that the required input and configuration folders are available in their expected locations.

3.3. Verifying the installation with the automated smoke test

PVMSim includes an automated smoke test that verifies whether the software, benchmark inputs, optimization routine, and reproducibility exports are working as expected. The test runs the STM6-40/36 benchmark curve using the fixed seed 42 and checks that the resulting RMSE_{obj} agrees with the reference value of $1.728401820819 \times 10^{-3}$ A within the configured numerical tolerance. It also verifies the generation of the required run artifacts and the presence of the expected source-code entries in `checksums.sha256`.

The test can be launched from the graphical interface by selecting `Tools > Run Smoke Test`, as shown in Figure 3.

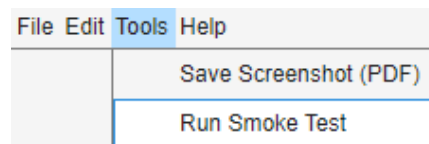


Figure 3. Tools menu used to launch the automated smoke test from the graphical interface.

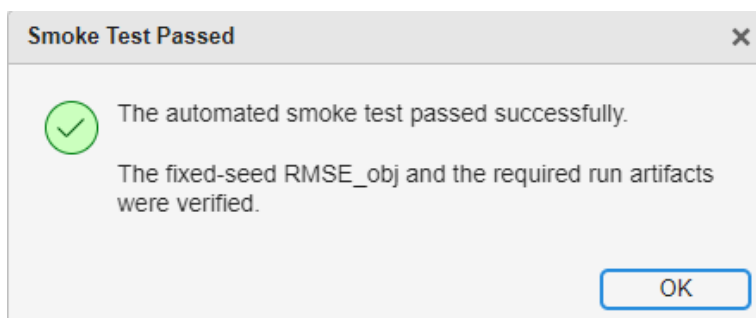
Alternatively, it can be executed from the MATLAB command window while the Current Folder is set to the PVMSim project root:

```
addpath(fullfile(pwd, 'tests'));
smoke_test;
```

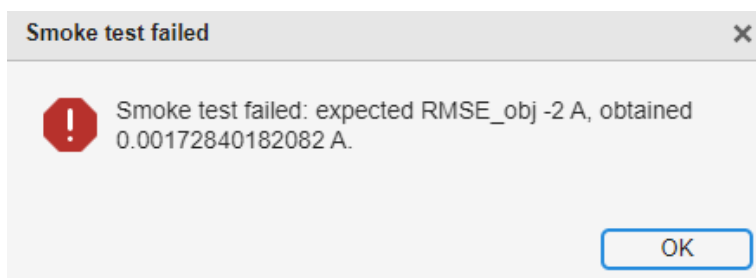
If all verification checks pass, the confirmation message shown in Figure 4a is displayed. If the computed objective value differs from the reference value, a required run artifact is missing, or a checksum entry cannot be verified, the error message in Figure 4b reports the failed check.

✓ Expected outcome

A successful smoke test should display the green confirmation message, reproduce the fixed-seed RMSE_{obj} within the configured tolerance, temporarily generate and verify the required run artifacts, and verify the expected entries in `checksums.sha256`. A failed test stops at the first unsuccessful check and displays the corresponding diagnostic message.



(a) Successful smoke test.



(b) Failed smoke test.

Figure 4. Smoke-test feedback displayed after the automated verification routine.

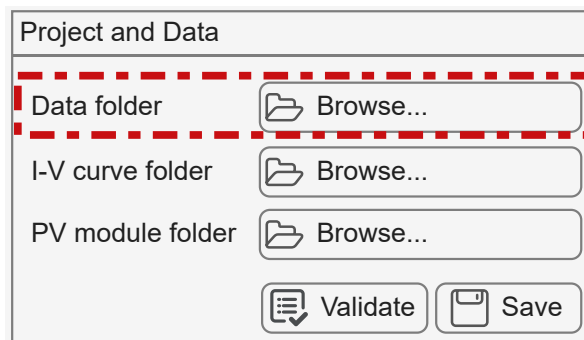
4. Step-by-Step Tutorial

4.1. Single-run workflow

The first successful run in *PVMSim* requires a valid project root, a readable PV module definition, and at least one measured I–V curve consistent with that module. The interface is designed so that the run buttons remain disabled until the workspace passes the validation step. The sequence below follows the actual logic implemented in the app.

4.1.1. Step 1. Set the project workspace and validate the default folders

Begin in the persistent left-side panel of the main window. The first required action is selecting the project root folder with the **Data folder** browser shown in Figure 5. This folder is expected to contain, by convention, the subfolders `data/iv` for measured I–V files and `config/modules` for PV module files.

**Figure 5.** Project root selector used to define the PVMSim workspace.

After a valid root folder is selected, the app can automatically infer the default I–V and module folders. For that reason, the controls in Figures 6 and 7 are optional in the standard repository layout and are mainly needed when the user wants to override the default locations.

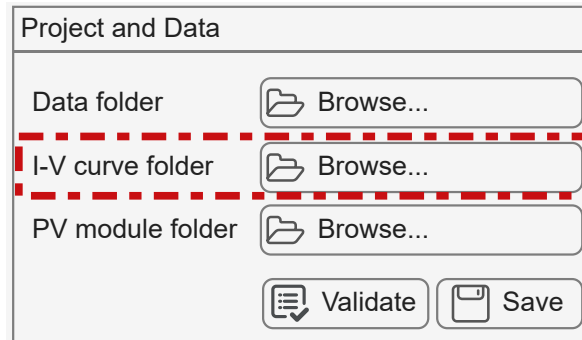


Figure 6. Optional selector for the measured I–V folder. This step is only needed when the curves are not stored in `<ProjectRoot>/data/iv`.

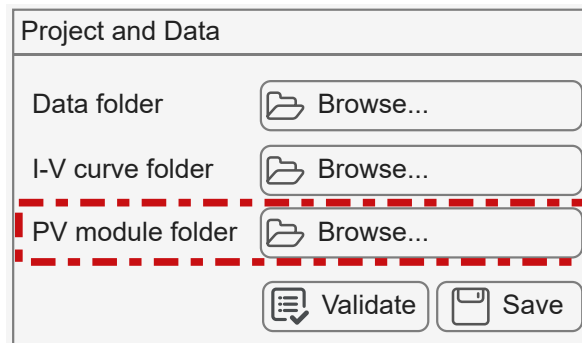


Figure 7. Optional selector for the PV module folder. This step is only needed when the module files are not stored in `<ProjectRoot>/config/modules`.

Once the folders are defined, press **Validate** as shown in Figure 8. Internally, the app checks that the project root exists, that the I–V folder contains at least one `.txt` curve file, that the module folder contains at least one valid module definition, and that at least one sampled file from each folder can be parsed correctly. A successful validation enables both **Run** and **Run Batch**. A failed validation keeps both execution buttons disabled.

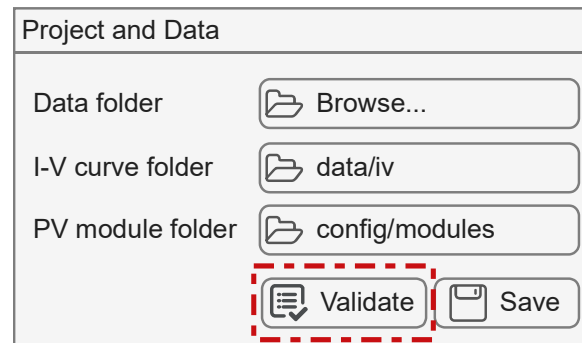
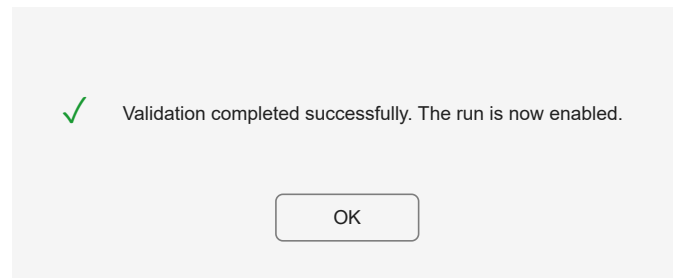
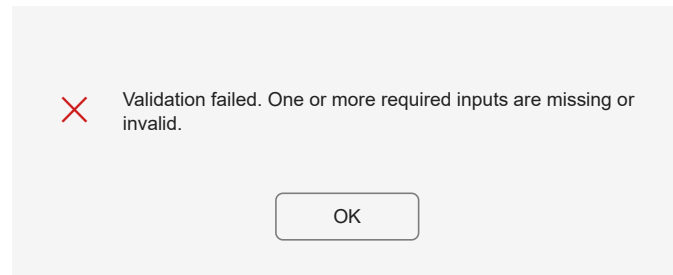


Figure 8. Validation button used to check the workspace before any model run.

If all checks pass, the success message shown in Figure 9a is displayed. Otherwise, the error message in Figure 9b identifies that the workspace is incomplete or inconsistent.



(a) Successful validation



(b) Failed validation

Figure 9. Validation feedback displayed after checking the project workspace.

✔ Expected outcome

If validation is successful, the three folder selectors should point to the intended workspace, the validation pop-up should confirm success, and the application log should record the detected project root, I–V folder, and module folder, as shown in Figure 8.

4.1.2. Step 2. Import the PV module electrical data

The PV module file is imported from the panel **PV Module Electrical Parameters (Datasheet)**. Press the **Import** button and select a valid **.txt** file from the module folder. The app reads the file, normalizes the field names, and fills the datasheet table with the quantities used later by the staged extraction routine.

PV Module Electrical Parameters (Datasheet)

	Value	Unit	Status	
Pmpp				
Voc				
Isc				
Vmpp				
Impp				
Ns				
Np				
Ki				
Kv				
NOCT				
Tref				
Gref				

↓

Import

↺

Reset

Figure 10. Import action for loading the PV module electrical data into the datasheet table.

When the import succeeds, the datasheet table should look like Figure 10. The rows correspond to the actual app fields Pmpp, Voc, Isc, Vmpp, Impp, Ns, Np, Ki, Kv, NOCT, Tref, Gref, CellLength, and CellWidth. Each row contains a numerical value, its unit, and a status label.

PV Module Electrical Parameters (Datasheet)				
	Value	Unit	Status	
Pmpp	NaN		MISSING	▲
Voc	21.0200	V	OK	
Isc	1.6630	A	OK	
Vmpp	16.9800	V	OK	
Impp	1.5000	A	OK	
Ns	36	cells	OK	
Np	NaN	strings	MISSING	
Ki	NaN	A/°C	OK	
Kv	NaN	V/°C	OK	
NOCT	NaN	°C	OK	
Tref	51	°C	OK	
Gref	1000	W/m ²	OK	▼
 Import  Reset				

Figure 11. Expected view of the PV module datasheet table after a successful import.

The content of this panel must be interpreted carefully. Numeric entries indicate that the quantity was successfully parsed from the module file. Entries shown as **NaN** indicate that the corresponding numerical value is not available in the imported file. A status label equal to **MISSING** means that the field could not be recovered and remains absent at that stage. This distinction is important because the algorithm requires, at minimum, physically valid **Ns** and **Isc** values before the extraction can start.

The **Reset** button in this panel clears the datasheet table, clears the cached paths in the interface, and disables **Run** and **Run Batch** until the workspace is validated again.

✓ Expected outcome

After a valid module file is imported, the datasheet table should be populated with the corresponding electrical quantities, the units should be visible in the second column, and the status column should no longer show unresolved entries for the quantities required by the algorithm, as shown in Figure 11.

⚠ Warning

Before starting the extraction, verify that **Isc**, **Ns**, and **Tref** contain physically valid values. Incorrect or missing entries in these rows may prevent a valid extraction setup or lead to non-physical fitted parameters.

4.1.3. Step 3. Import and select the measured I–V curve

Open the **Measured I–V Data** tab. Before importing any file, the tab is in the empty state shown in Figure 12. The plots contain no curves, the measured-curve table has no rows, and the

key-points table remains in the waiting state.

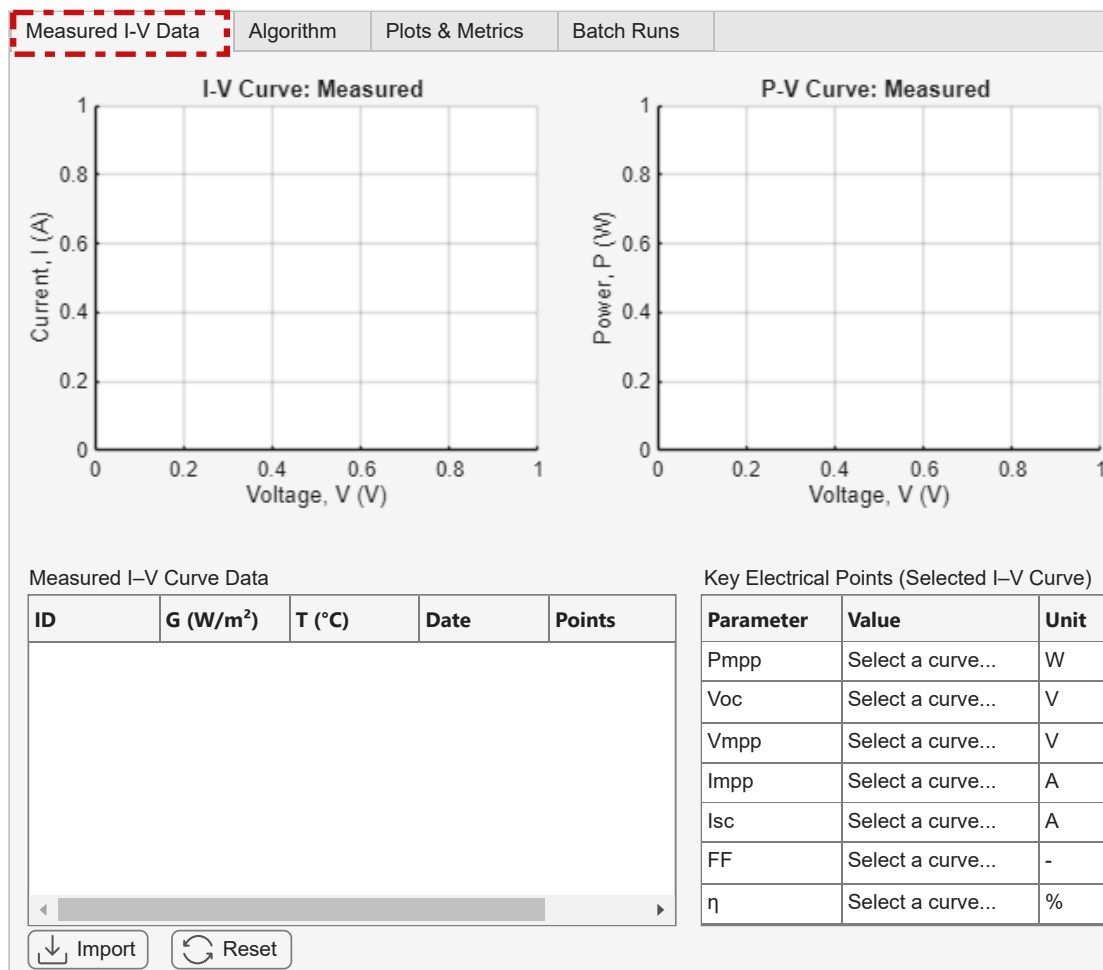


Figure 12. Initial empty state of the Measured I-V Data tab before importing any measured curve.

Press **Import** in this tab and select one or more measured **.txt** I-V files, as illustrated in Figure 13. The app reads the metadata stored in the file header, extracts the curve identifier, irradiance, temperature, date, and the number of available points, and appends each valid curve to the table. Duplicate identifiers are skipped, and invalid files are rejected with warning messages in the application log.

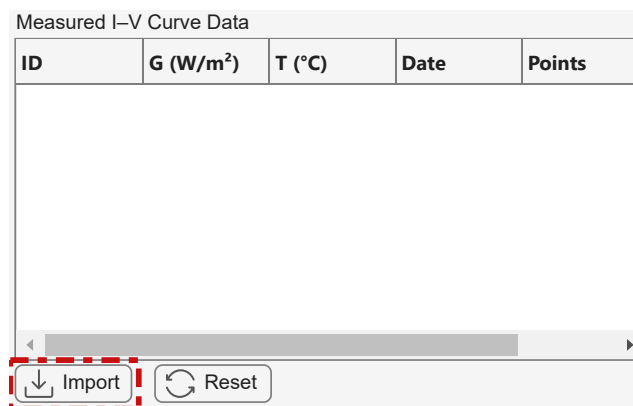


Figure 13. Import action for loading one or more measured I-V curves into the workspace.

After the files are imported, the user must select one curve from the table **Measured I-V Curve**

Data. The table columns are ID, G (W/m^2), T ($^{\circ}\text{C}$), Date, Points, and Valid. The selected row defines the curve used by both Run and Run Batch. Once a row is selected, the app updates the measured I–V plot, the measured P–V plot, and the key electrical points table with P_{mpp} , V_{oc} , V_{mpp} , I_{mpp} , I_{sc} , FF, and η for that measured dataset.

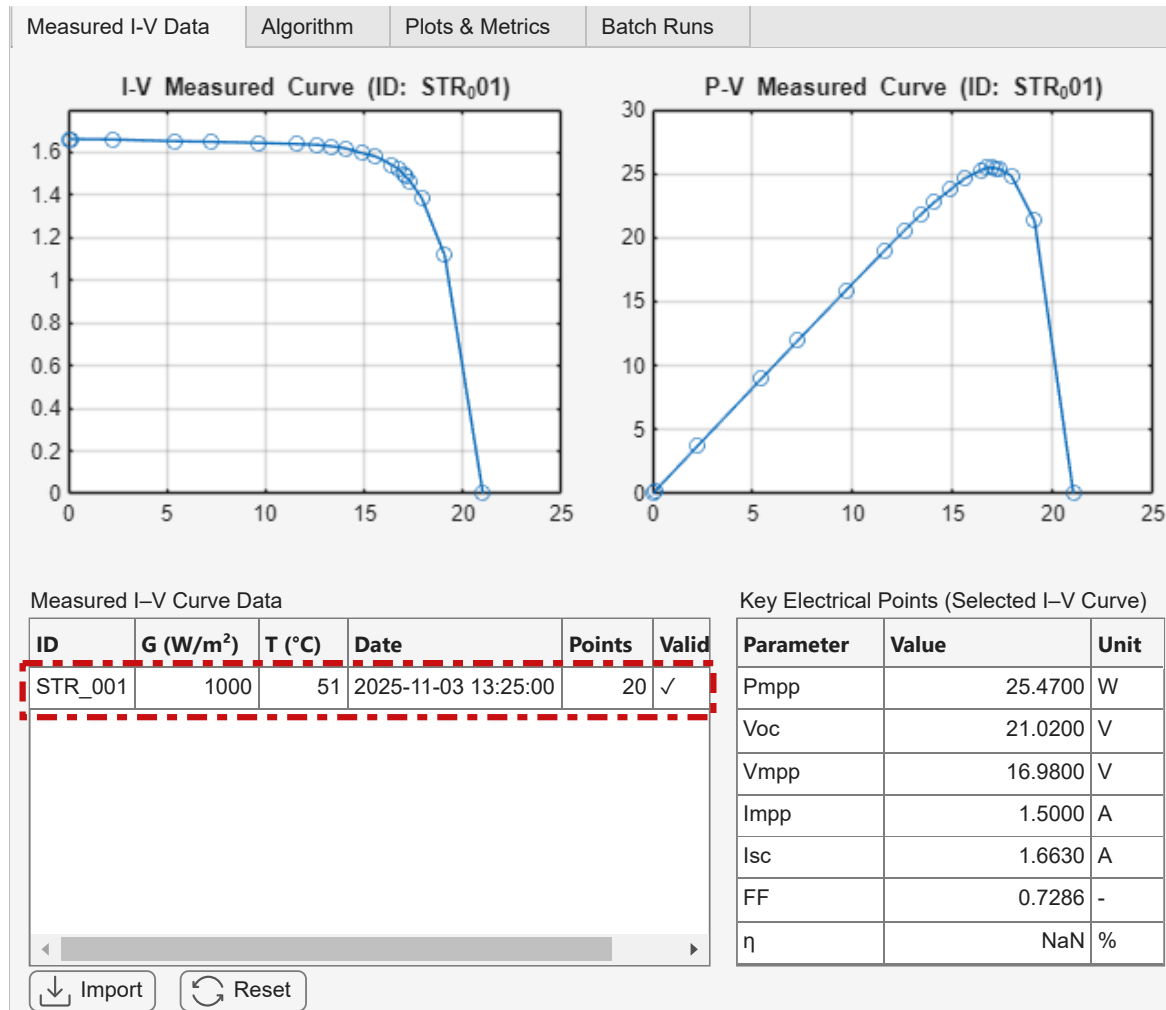


Figure 14. Expected state of the Measured I–V Data tab after importing several curves and selecting one valid measured dataset.

The **Reset** button in this tab clears the imported measured curves, empties the table, resets the selected-curve index, and restores the key-points table to its initial waiting state.

✓ Expected outcome

After a valid measured curve is imported and selected, the measured-curve table should contain at least one valid row, the selected curve should be displayed in both measured plots, and the key electrical points table should be filled with the seven quantities derived from that measured dataset, as shown in Figure 14.

i Note

After a measured curve is selected, the measured-curve table, the I–V plot, the P–V plot, and the key-points table are updated together to reflect the same dataset.

4.1.4. Step 4. Review the algorithm tab before execution

Move to the **Algorithm** tab. Before the first run, the tab should appear as in Figure 15. The extracted-parameters table is still in the waiting state, the convergence plot is empty, and the algorithm log has not yet recorded any optimization messages.

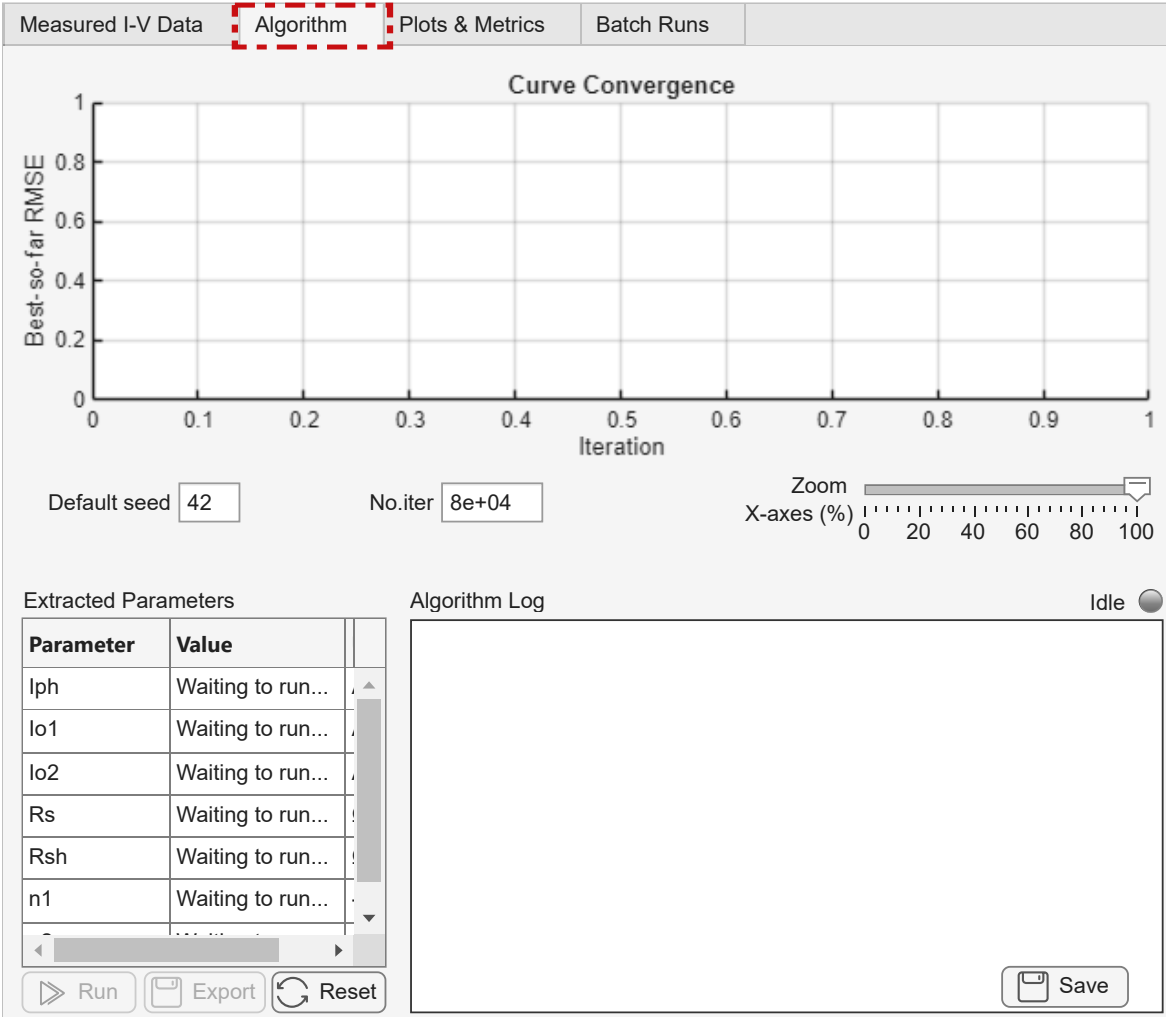


Figure 15. Initial empty state of the **Algorithm** tab before running the staged extraction routine.

Two numerical controls are available in this tab: **Default seed** and **Max Iteration**. In the current implementation, their default values are 42 and 80000, respectively. Both are already set when the app starts, so this step is optional for a standard run.

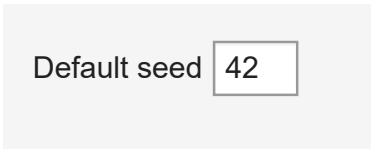


Figure 16. Seed control used to define the random seed for controlled reruns. The default value is 42.



Figure 17. Iteration-budget control used to define the maximum number of iterations for the staged extraction. The default value is 80000.

The **Zoom x axes** control does not change the extraction itself. It only changes how much of the convergence history is displayed in the **Curve Convergence** plot after a run. The default value is 100%, which displays the full available iteration range.

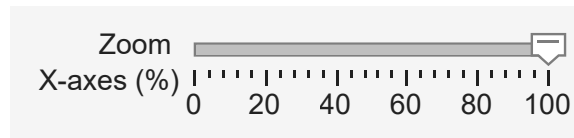


Figure 18. Zoom control used only to adjust the visible range of the convergence plot after a run.

✓ Expected pre-run state

Before execution, the tab should remain unfilled, the extracted-parameters table should still display waiting placeholders, and the controls should show the intended seed and iteration budget for the run.

4.1.5. Step 5. Run the staged parameter extraction

Press **Run** in the **Algorithm** tab, as shown in Figure 19. During execution, the app checks again that a measured curve has been imported and selected, and that the datasheet table contains valid values for the required module quantities. If those conditions are met, the staged extraction routine is launched with the selected measured curve and the current values of **Seed** and **Max Iteration**.

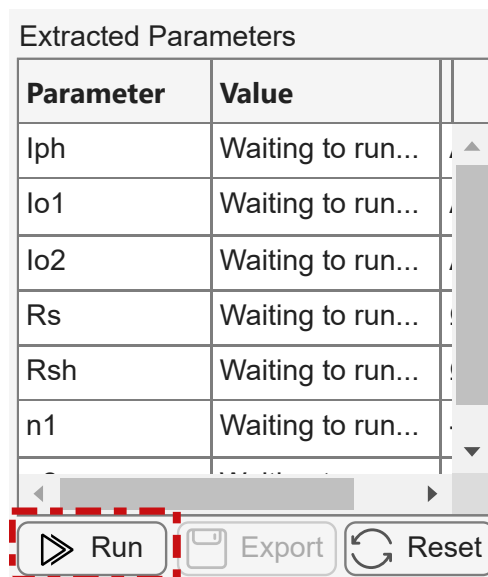


Figure 19. Run button used to start the staged extraction algorithm from the **Algorithm** tab.

While the algorithm is running, the status lamp turns yellow and the label changes to **Running**. At the same time, the value column of the extracted-parameters table and the KPI table is

temporarily replaced by the message **Calculating....** If the run finishes correctly, the lamp turns green and the label changes to **Done**. If an exception occurs, the lamp turns red and the label changes to **Error**.

Table 2. Status indicator used in the **Algorithm** tab.

Indicator	Label	Meaning
●	Idle	No extraction is currently running.
●	Running	The staged extraction is in progress.
●	Done	The extraction finished successfully.
●	Error	The extraction stopped because of an execution error.

If the run succeeds, the extracted-parameters table is filled with the seven parameters of the double-diode model:

$$[I_{ph}, I_{01}, I_{02}, R_s, R_{sh}, n_1, n_2]$$

The **Algorithm Log** area is also filled with time-stamped messages describing the execution, including the selected seed, iteration budget, and convergence-related messages generated during the run. Once a successful run exists, the parameter export button and the algorithm-log export button become useful post-processing actions.

The **Reset** button in this tab restores the extracted-parameters table to the waiting state and clears the convergence display for a new run configuration.

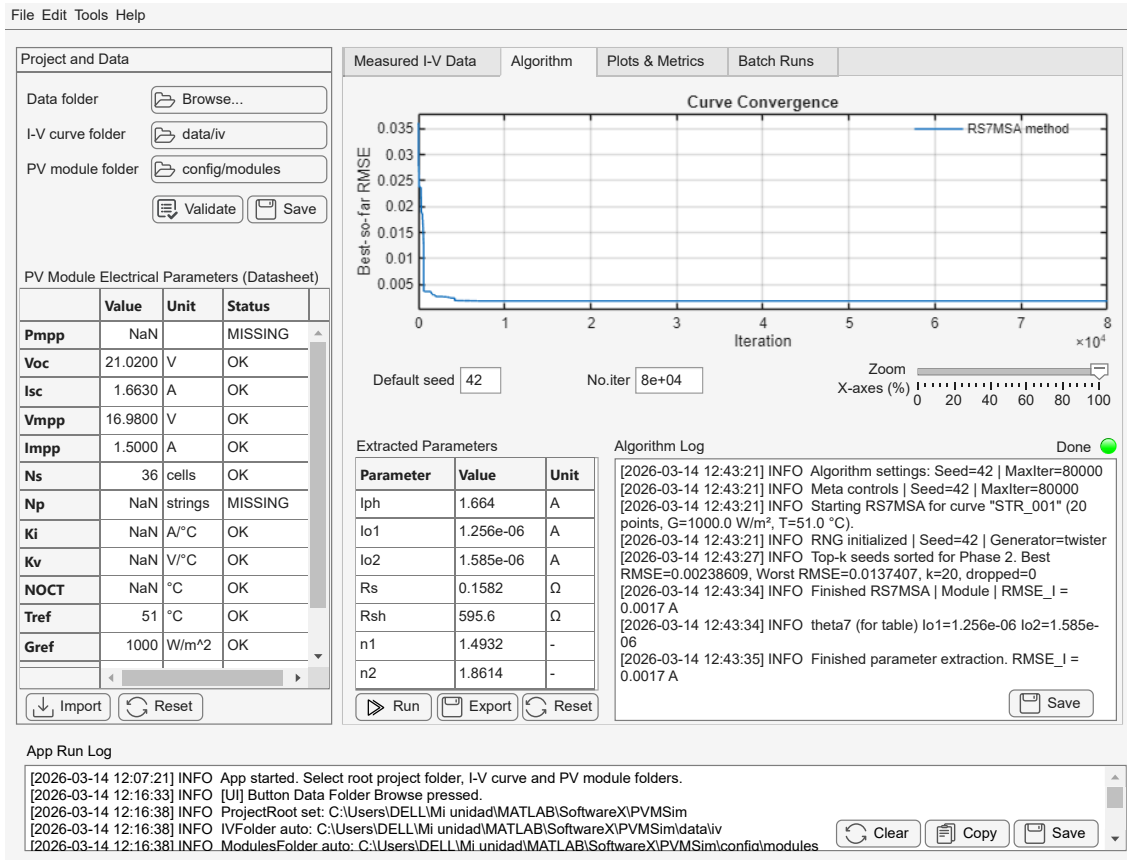


Figure 20. Expected state of the **Algorithm** tab after a successful staged extraction.

✓ Expected outcome

After a successful run, the status lamp should be green, the extracted-parameters table should contain numerical values for the seven model parameters, the convergence plot should display the best-so-far RMSE trajectory, and the algorithm log should contain the execution trace of the completed run.

i Note

The extracted-parameters table reports the fitted values of I_{ph} , I_{o1} , I_{o2} , R_s , R_{sh} , n_1 , and n_2 , which together define the calibrated double-diode model for the selected measured curve.

4.1.6. Step 6. Inspect plots and metrics after the run

Open the **Plots & Metrics** tab. Before running the algorithm, this tab is empty, as shown in Figure 21. The comparison plots contain no calculated curve, and the metrics table still shows waiting placeholders.

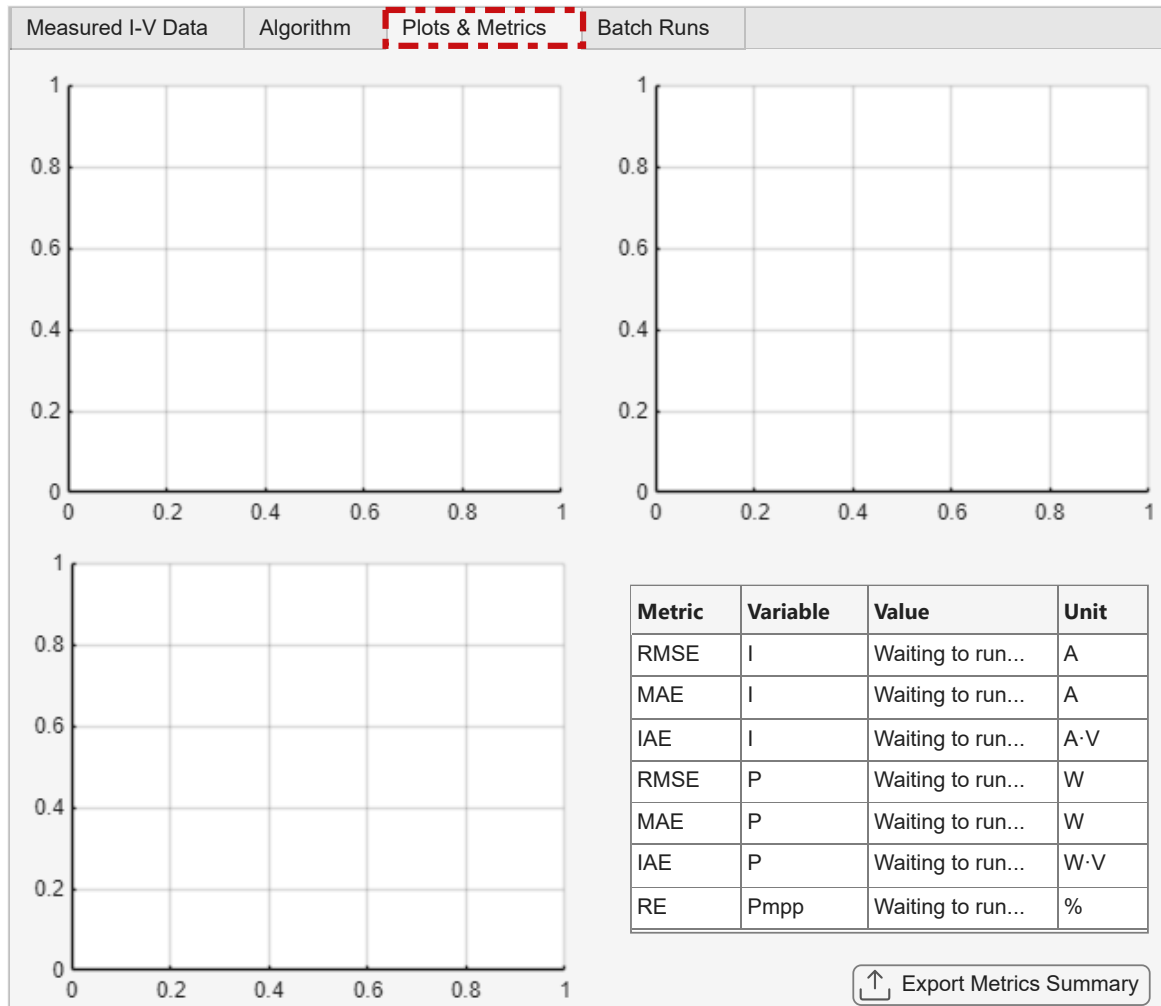


Figure 21. Initial empty state of the **Plots & Metrics** tab before parameter extraction.

After a successful run, this tab is filled automatically. The first comparison axis overlays the measured and calculated I-V curves. The second comparison axis overlays the measured and calculated P-V curves. The residual axis displays the current residuals as a function of voltage.

The KPI table is filled with the metrics actually used by the app: RMSE, MAE, and IAE for current; RMSE, MAE, and IAE for power; and RE for P_{mpp} . The export button in this tab becomes the natural way to save the metrics summary after the run.

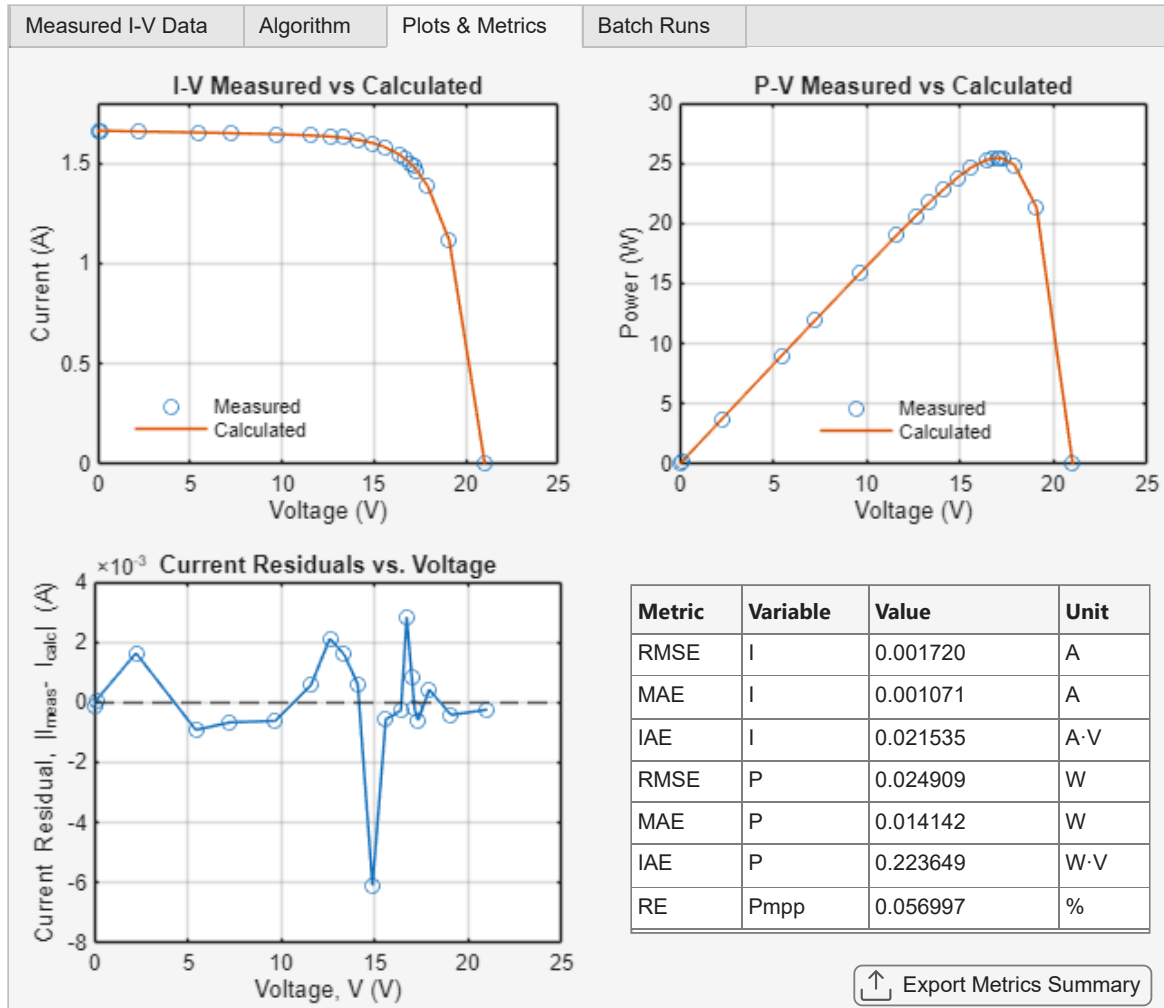


Figure 22. Expected state of the **Plots & Metrics** tab after a successful run, including curve comparison, residual analysis, and KPI summary.

✓ Expected outcome

After a successful run, the measured and calculated curves should overlap in the two comparison plots, the residual plot should be visible, the KPI table should be filled with numerical values, and the metrics summary should be ready for export from this tab.

i Note

The KPI table summarizes the main agreement metrics between the measured curve and the fitted model. These indicators help assess the quality of the parameter-extraction run from complementary perspectives.

4.2. Batch-run workflow

The **Batch** and **Report** tab is used after the single-run workflow has already been verified with a selected measured curve and an imported module datasheet. This mode repeats the extraction several times and summarizes run-to-run variability.

Before execution, the tab is in the empty state shown in Figure 23. The summary table and detailed-runs table are empty, and the batch status lamp remains in the idle state.

Measured I-V Data	Algorithm	Plots & Metrics	Batch Runs			
Run Status Table						
Run	Seed	RMSE obj	Time (s)	Status		
Batch Run Summary						
Model	Total ...	Best	Worst	Mean	Std	Time (s/run)
▶ Run Batch ⏹ Cancel 📄 Export Summary No. Runs 3 ☒ Consecutive Seeds Idle 						

Figure 23. Initial empty state of the Batch and Report tab before executing repeated runs.

In the current implementation, the batch controls are initialized with `N runs = 3` and `Use consecutive seeds = true`. The minimum accepted number of runs is 2. When batch execution starts, the lamp turns yellow and the label changes to **Running**. If the batch completes successfully, the lamp turns green and the label changes to **Done**. If validation fails or execution stops with an error, the lamp turns red and the label changes to **Failed**. The idle state is represented by a gray lamp.

Table 3. Status indicator used in the Batch and Report tab.

Indicator	Label	Meaning
	Idle	No batch execution is active.
	Running	The repeated-run procedure is in progress.
	Done	The batch finished and summary statistics were computed.
	Failed	The batch stopped because of invalid inputs or an execution error.

During execution, the detailed-runs table is filled run by run with the columns **Run**, **Seed**, **RMSE obj**, **Time (s)**, and **Status**. After completion, the summary table reports **Model**, **Total Runs**,

Best, Worst, Mean, Std, and Time (s/run). This tab therefore provides a compact view of stochastic variability across repeated extractions.

Measured I-V Data
Algorithm
Plots & Metrics
Batch Runs

Run Status Table

Run	Seed	RMSE obj	Time (s)	Status
9	50	1.785184e-03	15.283	OK
10	51	1.703262e-03	15.459	OK
11	52	1.743017e-03	13.681	OK
12	53	1.722761e-03	14.740	OK
13	54	1.741392e-03	13.890	OK
14	55	1.882237e-03	14.764	OK
15	56	1.760961e-03	14.612	OK
16	57	1.724208e-03	15.281	OK
17	58	2.152704e-03	13.382	OK
18	59	1.708341e-03	13.117	OK
19	60	1.744251e-03	13.177	OK
20	61	1.733486e-03	13.175	OK
21	62	1.736346e-03	13.101	OK

Batch Run Summary

Model	Total ...	Best	Worst	Mean	Std	Time (s/run)
Module	21	1.703262e-03	2.152704e-03	1.786472e-03	1.201977e-04	14.505

Run Batch
Cancel
Export Summary
No. Runs 21
☒ Consecutive Seeds
Done

Figure 24. Expected state of the Batch and Report tab after a successful repeated-run execution.

The **Export Summary** action writes the batch tables to an Excel workbook with separate sheets for the summary table and the per-run detail table. By default, the file is exported to `<ProjectRoot>/outputs/exports/`, which is the same export location used by the other output actions in the app.

✓ Expected outcome

If the batch setup and execution are completed successfully, the panel should appear as shown in Figure 24, with a green status lamp, aggregate statistics in the summary table, one row per repeated run in the detailed-runs table, and the export option ready to save the batch report.

5. Input Data and File Formats

5.1. Module definition files

Module definition files store the electrical and thermal parameters associated with the selected PV module. In the current project structure, these files are plain-text files using MATLAB-style assignments.

```
Ns = 36;  
Voc = 21.6;  
Isc = 7.84;  
Vmpp = 17.3;  
Impp = 7.23;
```

5.2. Measured I–V files

Measured I–V files are plain-text files that contain voltage–current pairs, typically in a two-column or comma-separated format consistent with the repository parser.

```
0.00, 7.84  
0.50, 7.82  
1.00, 7.81  
...
```

5.3. Optional metadata

Optional metadata may be provided in header lines. Typical fields include experiment identifier, irradiance, temperature, and date.

```
% ID=Curve_001  
% G=1000  
% T=25  
% Date=2026-03-12
```

6. Output Files and Run Exports

The exact set of output files depends on the execution mode and the current app implementation. The following files are representative of the structured run package described for the project.

6.1. log.txt

Text-based execution log capturing run events, messages, warnings, and execution status.

6.2. run_summary.csv

Comma-separated summary file containing the main metrics and selected run descriptors.

6.3. run_results.mat

MATLAB data file containing numerical outputs, extracted parameters, and run-level structures.

6.4. run_config.json

Machine-readable configuration snapshot storing the effective run settings, input references, and execution metadata.

6.5. checksums.sha256

Integrity file containing cryptographic hashes for selected outputs and, when configured, selected code files.

Table 4. Representative run exports used by PVMSim.

File	Purpose
log.txt	Execution trace and run messages
run_summary.csv	Summary metrics and selected scalar outputs
run_results.mat	Full MATLAB output structure
run_config.json	Input, configuration, and environment snapshot
checksums.sha256	Integrity verification for exported artifacts

7. Recommended Workflow

A practical workflow for routine use is:

1. Define the project root in the left-side project workspace; if the standard repository layout is used, the app can automatically detect the measured I–V folder and the PV module folder;
2. Manually set the measured I–V and PV module folders only when the default locations must be overridden;
3. Import the PV module definition and verify that the datasheet table is populated correctly;
4. Import one or more measured I–V curves in the **Measured I–V Data** tab;
5. Select the measured curve to be modeled and verify that the measured plots and key electrical points update consistently;
6. Press **Validate** to confirm that the configured workspace is valid and that both execution buttons are enabled;
7. Set the seed and maximum iteration budget in the **Algorithm** tab;
8. Run the staged extraction routine; if no curve is explicitly selected, the app uses the first imported curve;
9. Inspect the extracted parameters, convergence plot, measured-versus-calculated overlays, residual plot, and KPI summary;
10. Use the **Batch Runs** tab when repeated executions are required to evaluate run-to-run variability;
11. Export summaries, logs, parameters, and figures for traceable comparison and reporting.

✓ Recommended practice

For comparative studies, keep the MATLAB release, code version, seed policy, and input files fixed across runs. This reduces ambiguity when interpreting differences between parameter sets or fit metrics.

8. Troubleshooting

The app does not start

Verify that the project root is on the MATLAB path and that the startup script or app entry point exists in the expected location.

The module file is not parsed correctly

Inspect the syntax of the module definition file and confirm that parameter names and numeric values follow the expected format.

The measured I–V file is rejected

Check the delimiter, number of columns, numeric consistency, and presence of malformed header lines.

The run finishes with poor fit quality

Inspect the input data quality, operating-point consistency, seed, and iteration budget. Poor convergence may also indicate inconsistent module definitions or input preprocessing issues.

Batch runs show variability

This may be expected for stochastic optimization. Use controlled seeds and document the iteration budget and software version when reporting comparisons.

9. Relation to CLI execution

The GUI described in this manual supports interactive single-run and batch-run execution. The corresponding command-line entry point is `run_main.m`, which also supports both execution modes: when `batchN = 1`, it performs a single reproducible run, and when `batchN > 1`, it executes repeated runs with seed control.

10. References

Note

The illustrative PV module adopted in this manual is supported by benchmark data reported in the literature. Its datasheet parameters and associated I–V curves are available in [3, 4].

References

- [1] PVMSim Repository. Project source code and documentation. <https://github.com/pvmsim>
- [2] MathWorks. MATLAB documentation. <https://www.mathworks.com/help/>
- [3] Li, S., Gong, W., Yan, X., Hu, C., Bai, D., Wang, L., and Gao, L. Parameter extraction of photovoltaic models using an improved teaching-learning-based optimization. *Energy Conversion and Management*, 186, 293–305, 2019. doi: 10.1016/j.enconman.2019.02.048.

- [4] Bakır, H. Elevating PV model performance: Accurate and reliable parameter extraction of solar cell models with state-of-art metaheuristic algorithms. *Next Energy*, 11, 100513, 2026. doi: 10.1016/j.nxener.2026.100513.